

Laboratorio 1**Esquemas de comunicación e introducción a Wireshark****Reporte Individual — Segunda Parte**

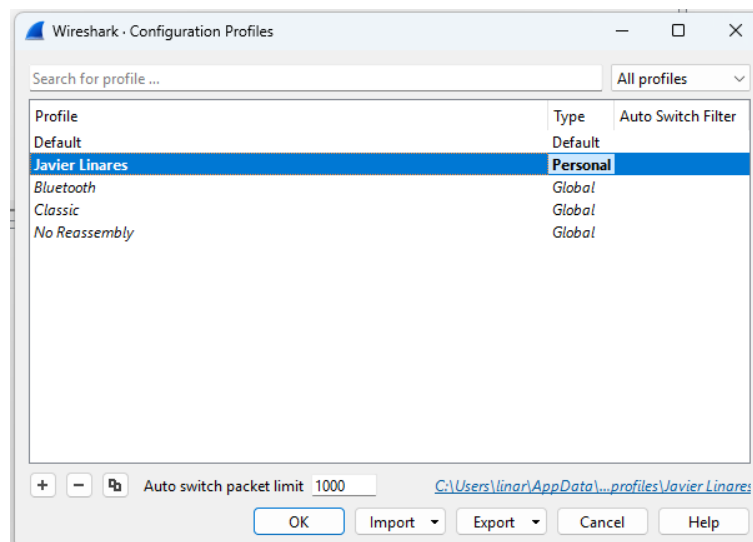
Javier Alexander Linares Chang — Carné: 231135

1. Introducción

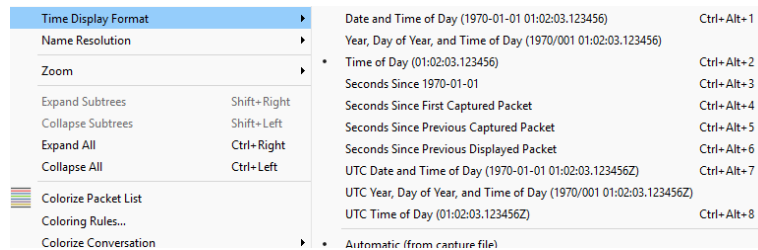
Este reporte cubre la segunda parte del Laboratorio 1, donde se trabajó con Wireshark para capturar y analizar tráfico de red. Se divide en tres partes: la personalización del entorno de Wireshark (perfil, columnas, colores y filtros), la configuración de una captura con ring buffer, y el análisis de una comunicación HTTP real. En cada sección se incluyen las capturas de pantalla que evidencian el trabajo realizado junto con las respuestas a las preguntas planteadas en la guía.

2. Personalización del entorno (Sección 3.4)

Se inició Wireshark y se creó un perfil de configuración con el nombre "Javier Linares", visible en la barra de estado inferior durante toda la práctica. Sobre el archivo intro-wireshark-trace1.pcap se aplicaron las siguientes personalizaciones.

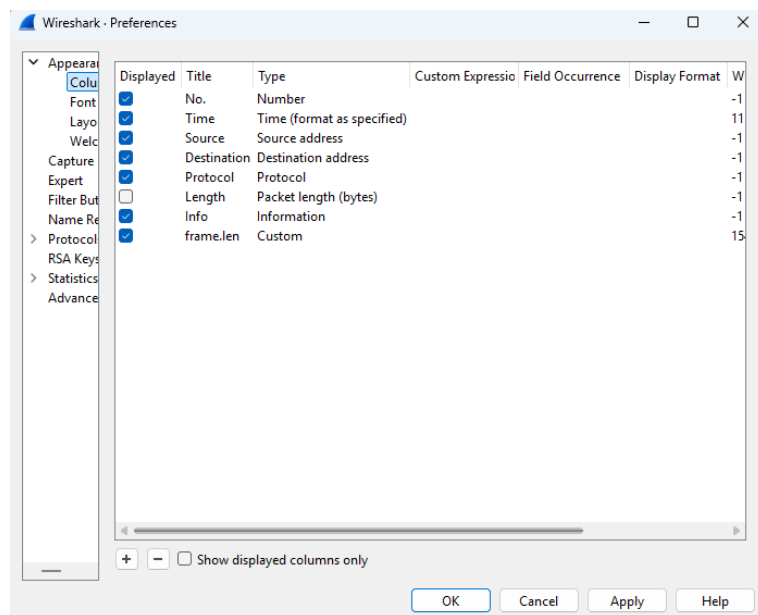
**2.1 Formato de tiempo Time of Day**

Se aplicó el formato de tiempo "Time of Day" (View → Time Display Format), por lo que la columna Time muestra la hora real del día en lugar de los segundos transcurridos desde el inicio de la captura.



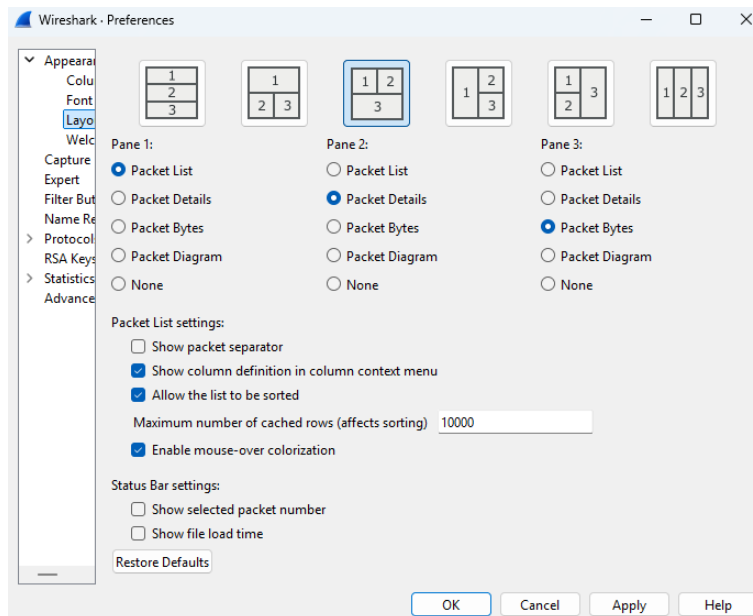
2.2 Columna de longitud del protocolo

Se agregó una columna personalizada con el campo frame.len al final de la lista de columnas, y se ocultó la columna "Length" que trae Wireshark por defecto.



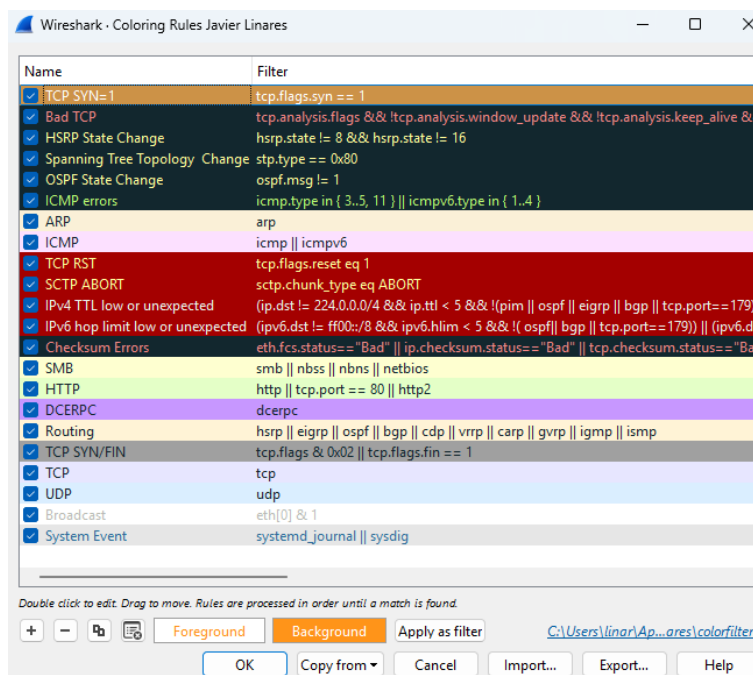
2.3 Distribución de paneles (Layout)

Se cambió la distribución de paneles a un esquema distinto al predeterminado: en lugar de los tres paneles apilados verticalmente, se dejó la lista de paquetes a la izquierda y los paneles de detalle y de bytes hexadecimales lado a lado a la derecha.



2.4 Regla de color y botón de filtro

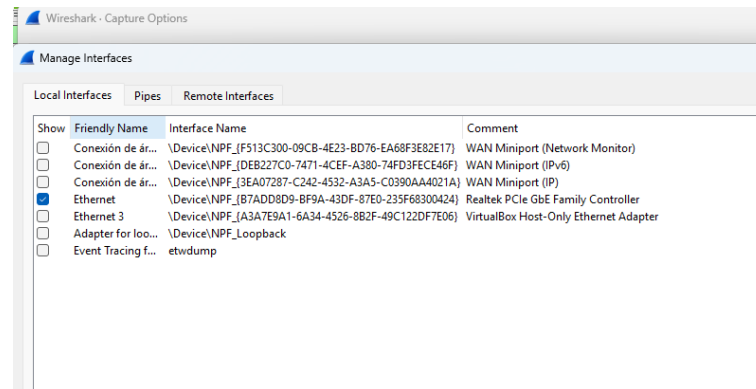
Se creó una regla de color llamada "TCP SYN=1" con el filtro `tcp.flags.syn == 1`, ubicada como primera regla de la lista para que se evalúe antes que la regla genérica de TCP, con un color de fondo naranja. También se creó un botón de acceso rápido llamado "SYN" con ese mismo filtro. En la vista principal se observan los paquetes SYN y SYN-ACK resaltados en naranja, con el botón "SYN" activo en la barra de filtros.



2.5 Lista simplificada de interfaces

Se ocultaron las interfaces virtuales del equipo (los WAN Miniports, el adaptador Host-Only de VirtualBox, el adaptador de loopback y el lector de Event Tracing for

Windows) mediante Capture → Options → Manage Interfaces, dejando visible únicamente la interfaz de red física real ("Ethernet", Realtek PCIe GbE Family Controller).



3. Configuración de la captura de paquetes (Sección 3.5)

3.1 Comando ipconfig/ifconfig

Se ejecutó el comando ipconfig en la terminal de Windows para revisar la configuración de red del equipo. La salida mostró dos adaptadores:

- Ethernet 3: dirección IPv4 192.168.56.1, máscara 255.255.255.0, sin puerta de enlace. Este es el adaptador Host-Only que crea VirtualBox (el mismo que se ocultó en Wireshark en la sección 2.5); al no tener gateway, no es un adaptador con salida real a internet, solo sirve para la comunicación entre el host y las máquinas virtuales.
- Ethernet: dirección IPv4 192.168.50.158, máscara 255.255.255.0, puerta de enlace 192.168.50.1. Este sí es el adaptador físico real con el que el equipo sale a internet, y es el mismo que se dejó visible como única interfaz en Wireshark.

Esto confirma que la interfaz que se debe usar para las capturas es "Ethernet" (la que tiene gateway configurado), mientras que "Ethernet 3" solo es tráfico interno de VirtualBox.

```
C:\Users\linar>ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet 3:

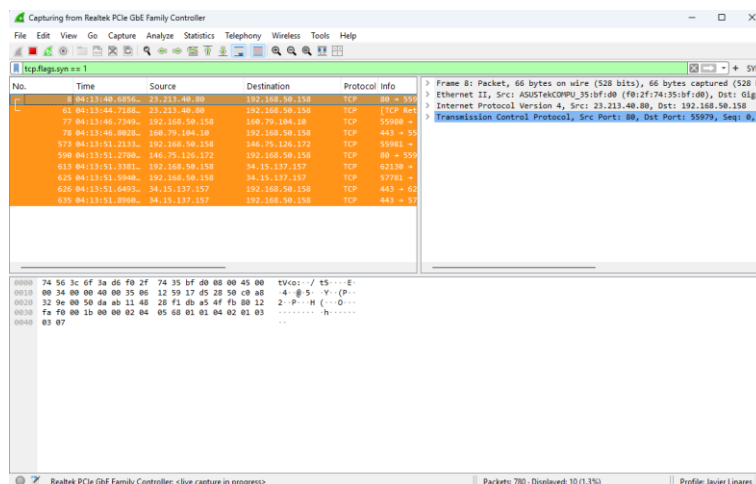
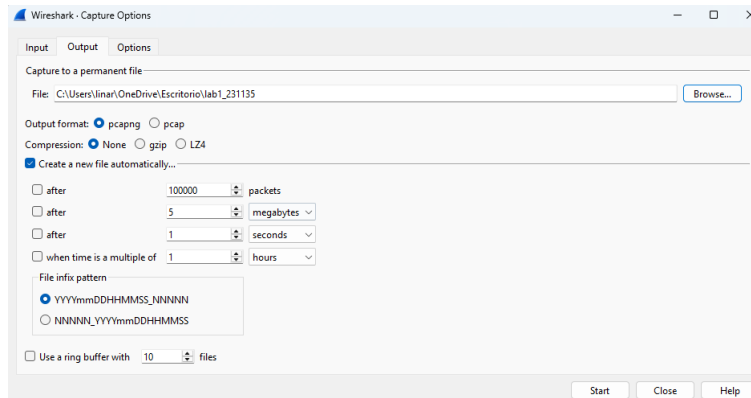
    Sufijo DNS específico para la conexión. . . :
    Vínculo dirección IPv6 local. . . : fe80::21ae:ad5b:ed86:e90d%11
    Dirección IPv4. . . . . : 192.168.56.1
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . :

Adaptador de Ethernet Ethernet:

    Sufijo DNS específico para la conexión. . . :
    Vínculo dirección IPv6 local. . . : fe80::82b3:f92:7c12:c3cd%12
    Dirección IPv4. . . . . : 192.168.50.158
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . : 192.168.50.1
```

3.2 Captura con ring buffer

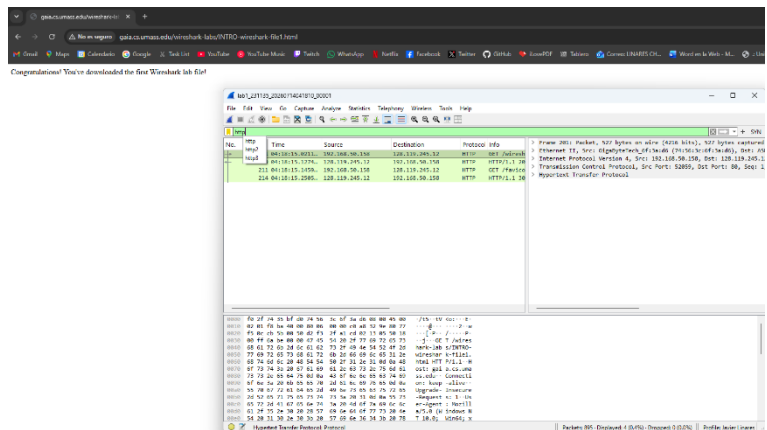
Se configuró una captura sobre la interfaz real con un ring buffer: tamaño máximo de 5 MB por archivo y un máximo de 10 archivos, con el nombre lab1_231135 (Capture → Options → Output).



lab1_231135_20260714041206_00245
lab1_231135_20260714041207_00246
lab1_231135_20260714041209_00247
lab1_231135_20260714041211_00248
lab1_231135_20260714041213_00249
lab1_231135_20260714041215_00250
lab1_231135_20260714041216_00251
lab1_231135_20260714041218_00252
lab1_231135_20260714041220_00253
lab1_231135_20260714041222_00254
lab1_231135_20260714041330_00001

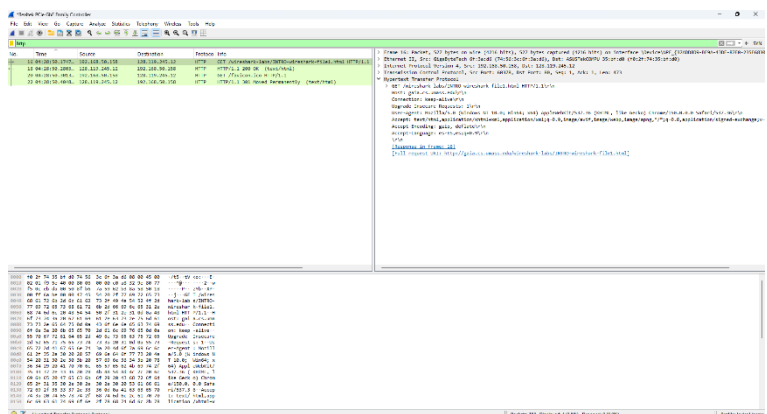
4. Análisis de paquetes HTTP (Sección 3.6)

Se limpió el caché del navegador, se inició una captura sin filtro sobre la interfaz Ethernet y se accedió a <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>. Al detener la captura y filtrar por http quedaron visibles la solicitud GET y la respuesta 200 OK de la página, además de una segunda solicitud a /favicon.ico que el navegador pide aparte y que no se tomó en cuenta para el análisis.



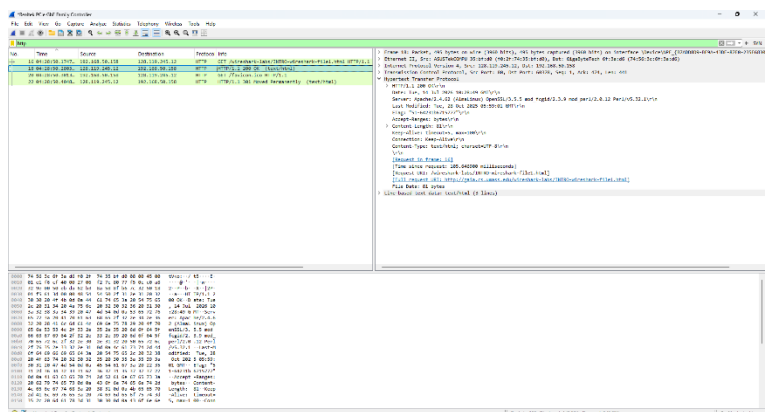
4.1 Solicitud del navegador (GET)

Al desplegar el bloque Hypertext Transfer Protocol del paquete GET se observa la solicitud GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1, dirigida a Host: gaia.cs.umass.edu, con User-Agent Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36, y la cabecera Accept-Language: es-ES,es;q=0.9.



4.2 Respuesta del servidor (200 OK)

El paquete de respuesta muestra HTTP/1.1 200 OK, con Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.5 mod_perl/2.0.12 Perl/v5.32.1, Content-Type: text/html; charset=UTF-8, y Content-Length: 81.



4.3 Respuestas a las preguntas

a) **¿Qué versión de HTTP está ejecutando su navegador?** HTTP/1.1, según la línea "GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1" de la solicitud.

b) **¿Qué versión de HTTP está ejecutando el servidor?** También HTTP/1.1, según la línea "HTTP/1.1 200 OK" de la respuesta.

c) **¿Qué lenguajes indica el navegador que acepta el servidor?** Según la cabecera Accept-Language: es-ES,es;q=0.9, el navegador indica que acepta español de España y español en general.

d) **¿Cuántos bytes de contenido fueron devueltos por el servidor?** 81 bytes, según la cabecera Content-Length: 81 de la respuesta.

e) **En caso de un problema de rendimiento, ¿en qué elementos de la red convendría escuchar los paquetes? ¿Es conveniente instalar Wireshark en el servidor?** Convendría capturar en varios puntos del recorrido: en el propio equipo (para descartar que el problema sea local, del navegador o de la tarjeta de red), en el router/gateway de la red local, y si se pudiera en algún punto intermedio hacia el servidor, para comparar en qué tramo se generan los retrasos o pérdidas. Instalar Wireshark en el servidor normalmente no es buena idea: casi nunca se tiene acceso administrativo a un servidor de terceros como gaia.cs.umass.edu, y aunque se tuviera, correr un analizador de paquetes ahí consume recursos del servidor y puede afectar su rendimiento y su seguridad. Es mejor capturar en el cliente y en los puntos de la red que sí están bajo nuestro control.

5. Discusión, hallazgos y comentarios

Esta práctica se sintió como el complemento lógico de la primera parte del laboratorio: ahí estuvimos codificando y decodificando mensajes a mano con Morse y Baudot, viendo de primera mano cómo aparecen los errores cuando uno transmite información, y acá con Wireshark se ve exactamente lo mismo pero automatizado y a la escala real de una red, con paquetes, direcciones y protocolos en vez de pulsos y bits contados a mano.

Lo que más me llamó la atención fue todo lo que viaja en texto plano dentro de una solicitud HTTP: el User-Agent completo del navegador, la versión, hasta el idioma que uno tiene configurado (Accept-Language). Uno normalmente no piensa en que cada vez que entra a una página está mandando esa información sin cifrar, visible para cualquiera que esté escuchando el tráfico.

También me costó al inicio entender por qué no me aparecía tráfico HTTP la primera vez que entré a la página del laboratorio: resulta que el navegador ya tenía la página guardada en caché de una visita anterior, entonces no se generó ninguna solicitud nueva al servidor. Tuve que limpiar el caché para forzar que el navegador volviera a pedirle la página al servidor y ahí sí pude capturar la comunicación completa.

La parte de personalizar el entorno (perfil, columnas, reglas de color, botón de filtro) al principio parece un paso extra innecesario, pero después se nota la diferencia: con 651 paquetes en una sola captura, tener los SYN resaltados en un color y un botón para filtrarlos de un clic ahorra bastante tiempo comparado con estar buscando manualmente. Lo mismo con el ring buffer, que evita que la captura llene el disco si se deja corriendo mucho tiempo, sobrescribiendo los archivos más viejos automáticamente.

6. Conclusiones

Wireshark deja ver, con datos reales, todo lo que normalmente pasa desapercibido cuando uno navega: direcciones, protocolos, encabezados y hasta el contenido cuando no va cifrado, lo cual refuerza en la práctica lo que se ve en teoría sobre cómo viajan los paquetes.

Personalizar el entorno (perfiles, columnas, reglas de color, botones de filtro y el layout) no es solo estética, hace mucho más rápido encontrar lo que uno busca cuando hay cientos de paquetes capturados.

El análisis de HTTP mostró que datos como la versión del protocolo, el idioma que acepta el navegador y el tamaño del contenido se pueden leer directamente del tráfico, y que HTTP no cifra nada de esto.

El caché del navegador puede hacer que no se capture tráfico nuevo si la página ya se había visitado antes, así que hay que limpiarlo para forzar una solicitud real al servidor.

Para diagnosticar problemas de red conviene capturar en el cliente y en los puntos intermedios bajo nuestro control, no en el servidor, ya que normalmente no se tiene acceso a él y hacerlo podría afectar su funcionamiento.

7. Referencias

Universidad del Valle de Guatemala. (2026). Laboratorio 1 – Esquemas de comunicación e introducción a Wireshark. Departamento de Ciencias de la Computación, CC3067 Redes.

Wireshark Foundation. (s. f.). Wireshark User's Guide. Recuperado de <https://www.wireshark.org/docs/>

Kurose, J. F. & Ross, K. W. Wireshark Labs: Getting Started (gaia.cs.umass.edu/wireshark-labs).